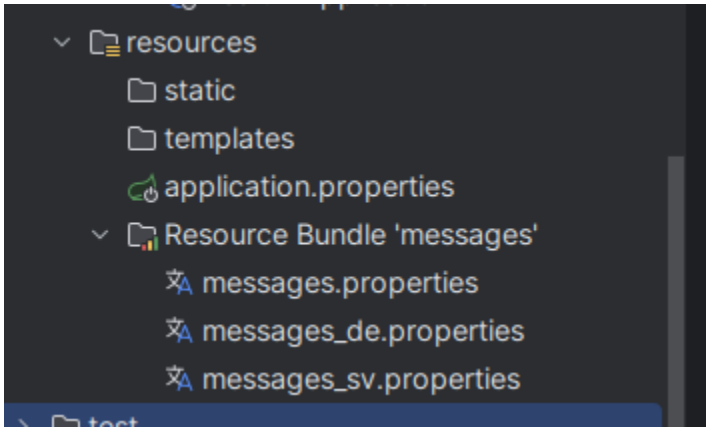
Assignment - RestFul Web Services - Part 2

Q1)Internationalization

A) Add support for Internationalization in your application allowing messages to be shown in English, German and Swedish, keeping English as default.

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-hateoas</artifactId>
</dependency>
```




```
greeting.message=Hello {0}
```

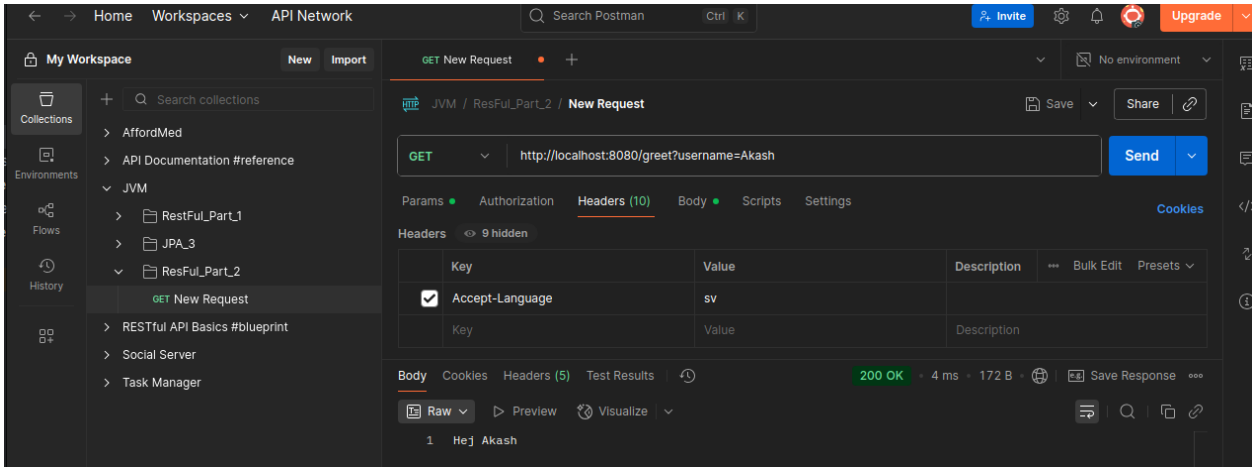
```
greeting.message=Hallo {0}
```

```
greeting.message=Hej {0}
```

B) Create a GET request which takes "username" as param and shows a localized message "Hello Username". (Use parameters in message properties)

```
import org.springframework.context.MessageSource;
import org.springframework.web.bind.annotation.*;

import java.util.Locale;

@RestController
public class GreetingController {
    @Autowired
    private MessageSource messageSource;

    @GetMapping("/greet")
    public String greet(@RequestParam String username, @RequestHeader(name = "Accept-Language", required = false) Locale locale) {
        return messageSource.getMessage( code: "greeting.message", new Object[]{username}, locale);
    }
}
```

```
com.akash.resful_2.controller.GreetingController
@Autowired
private MessageSource messageSource
```

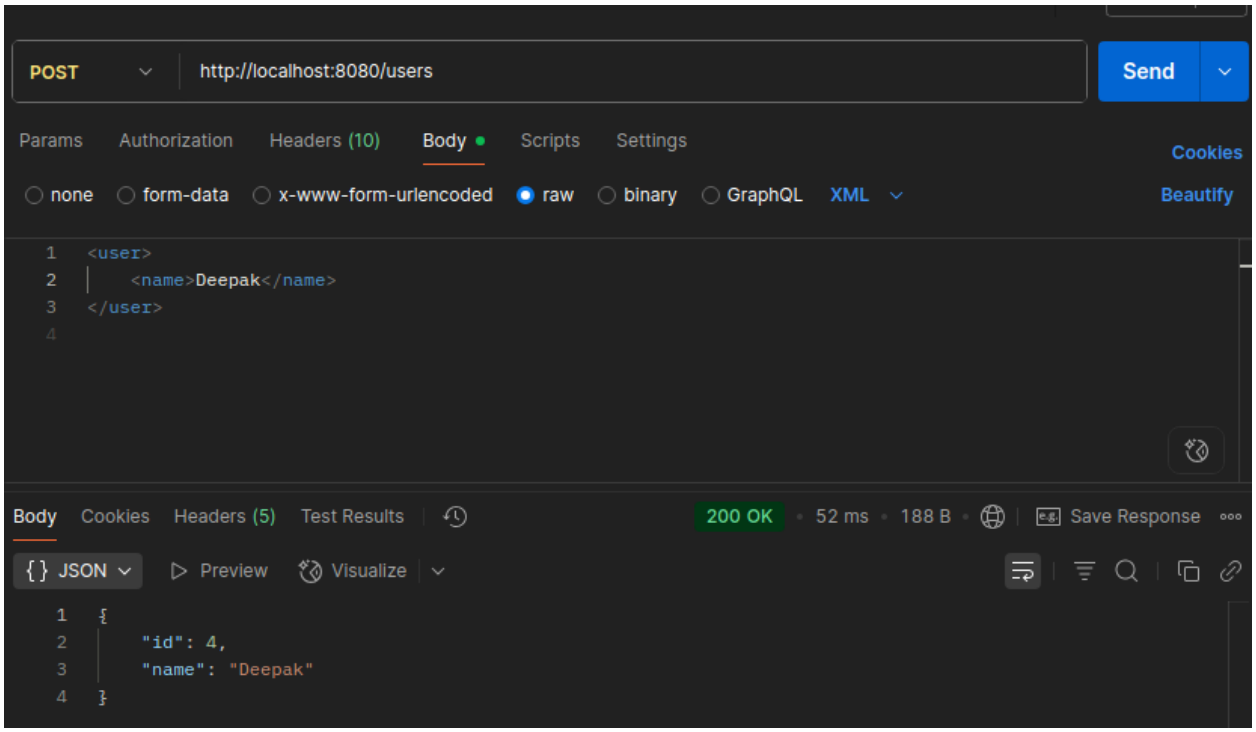


```java
import java.util.Locale;

@Configuration
public class I18nConfig {
    @Bean
    public LocaleResolver localeResolver() {
        AcceptHeaderLocaleResolver resolver = new AcceptHeaderLocaleResolver();
        resolver.setDefaultLocale(Locale.ENGLISH);
        return resolver;
    }
}
```

```properties
spring.application.name=Resful_2
spring.messages.basename=messages
spring.web.locale-resolver=fixed
spring.web.locale=en
```

Q2)Content Negotiation

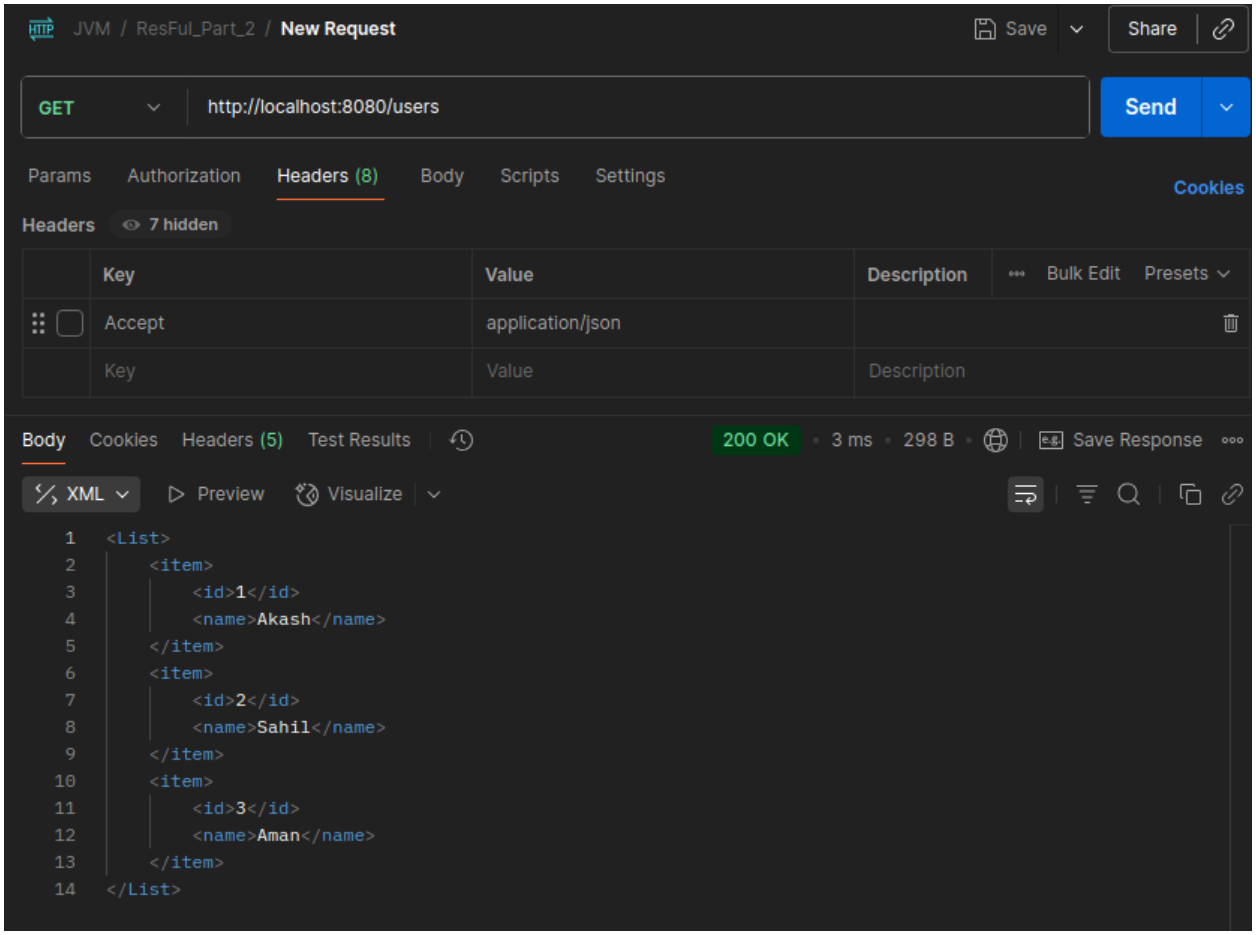
A) Create POST Method to create user details which can accept XML for user creation.




```java
@RestController
@RequestMapping("/users")
public class UserController {

    private final UserService userService;  3 usages

    public UserController(UserService userService) {
        this.userService = userService;
    }

    // A) POST - Accept XML to create user
    @PostMapping(consumes = { MediaType.APPLICATION_XML_VALUE, MediaType.APPLICATION_JSON_VALUE })
    public User createUser(@RequestBody User user) {
        return userService.createUser(user);
    }
```

```java
import com.fasterxml.jackson.annotation.JsonPropertyOrder;
import com.fasterxml.jackson.dataformat.xml.annotation.JacksonXmlRootElement;

@JacksonXmlRootElement(localName = "User") 12 usages
@JsonPropertyOrder({ "id", "name"})
public class User {
       private Long id; 3 usages

    private String name; 3 usages

    public User(Long id, String name) { 3 usages
        this.id = id;
        this.name = name;
    }
```

B) Create GET Method to fetch the list of users in XML format.

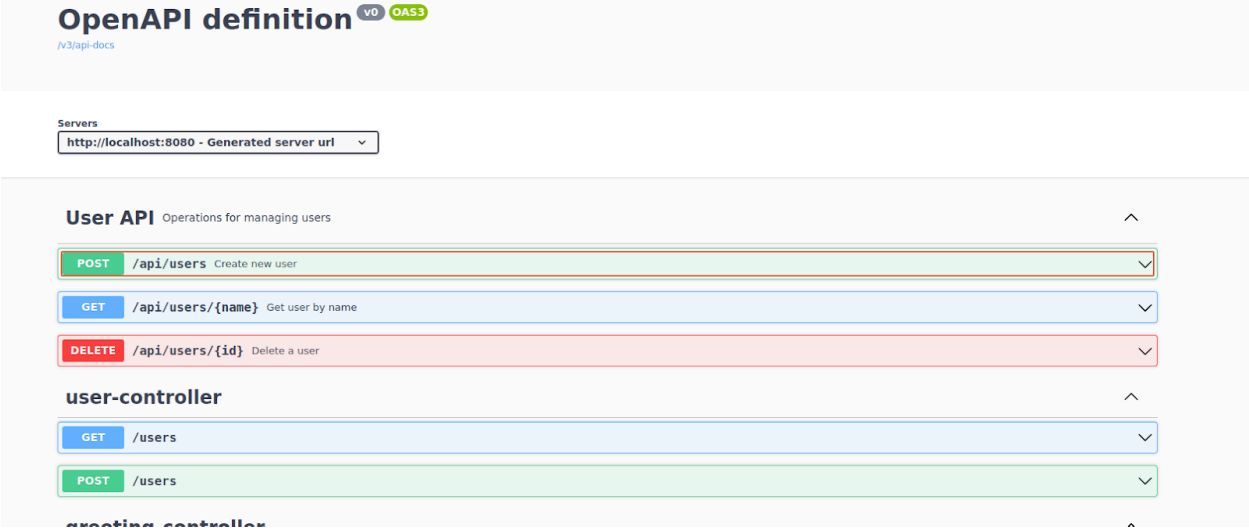


```
        return userService.createUser(user);
    }

    // B) GET - Return XML or JSON list of users
    @GetMapping(produces = { MediaType.APPLICATION_XML_VALUE, MediaType.APPLICATION_JSON_VALUE })
    public List<User> getAllUsers() {
        return userService.getAllUsers();
    }
}
```

Q3)Swagger

A) Configure the swagger plugin and create a document of the following methods: Get details of the User using GET request. Save details of the user using POST request. Delete a user using DELETE request.

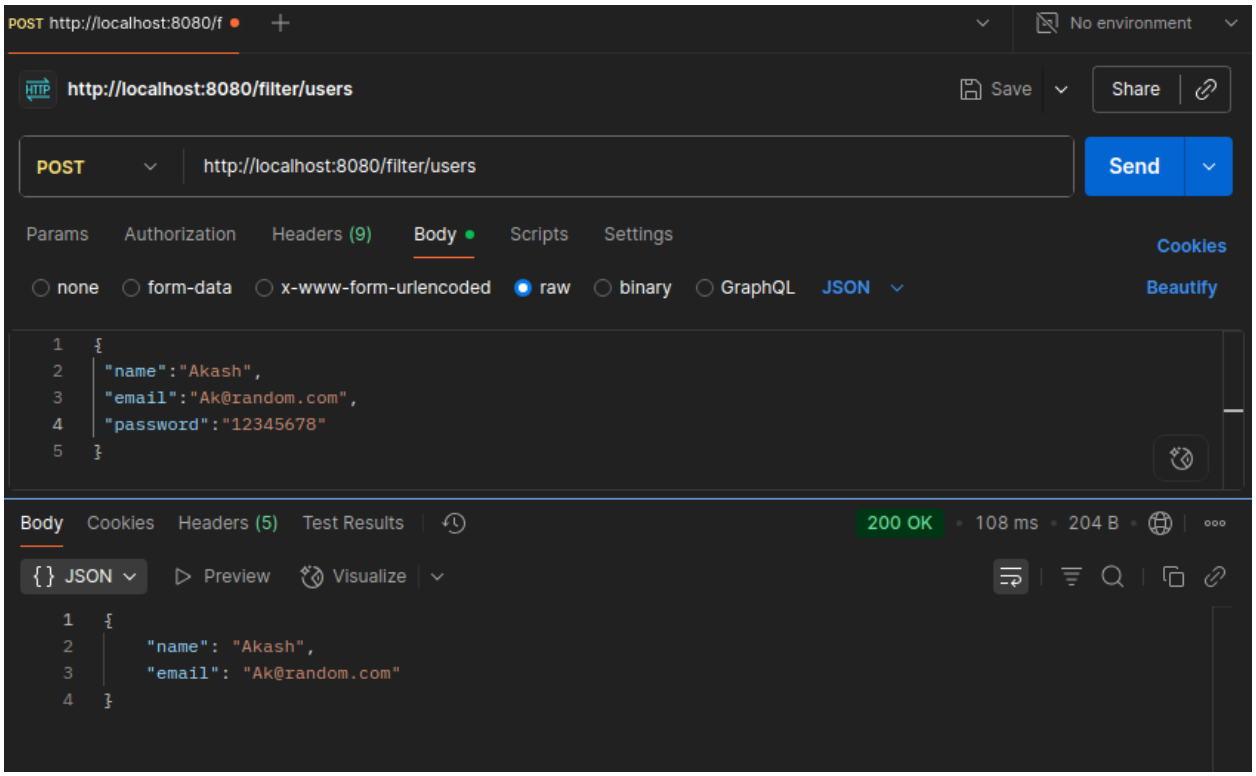
B) In swagger documentation, add the description of each class and URI so that in swagger UI the purpose of class and URI is clear.



```java
public class SwaggerUserController {

    private final UserService userService;  4 usages

    public SwaggerUserController(UserService userService) {
        this.userService = userService;
    }

    @Operation(summary = "Get user by name")
    @GetMapping("/{name}")
    public User getUser(@PathVariable String name) {
        return userService.getAllUsers().get(0);
    }

    @Operation(summary = "Create new user")
    @PostMapping
    public User saveUser(@RequestBody User user) {
        return userService.createUser(user);
    }

    @Operation(summary = "Delete a user")
    @DeleteMapping("/{id}")
    public User deleteUser(@PathVariable int id) {
        return userService.deleteUserByID(id);
    }
}
```

Q4)Static and Dynamic filtering

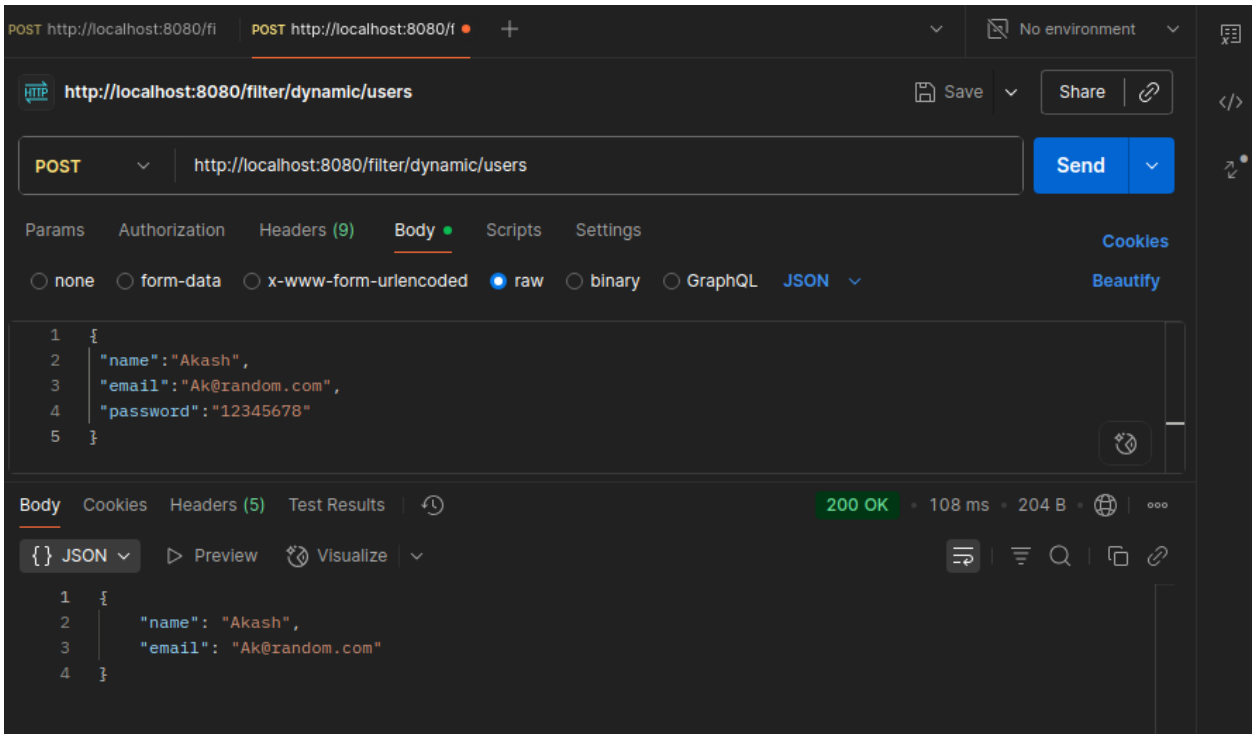
A) Create API which saves details of User (along with the password) but on successfully saving returns only non-critical data. (Use static filtering)



```
@RestController
@RequestMapping("/filter")
public class FilterController {


    @PostMapping("/users")
    public UserEntity createUser(@RequestBody UserEntity user) {
        return user;
    }
}
```

```java
public class UserEntity {  no usages

    private String name;  3 usages
    private String email;  3 usages

    @JsonIgnore  3 usages
    private String password;

    public UserEntity(String name, String email, String password) {  no usages
        this.name = name;
        this.email = email;
        this.password = password;
    }
```

B) Create another API that does the same by using Dynamic Filtering.

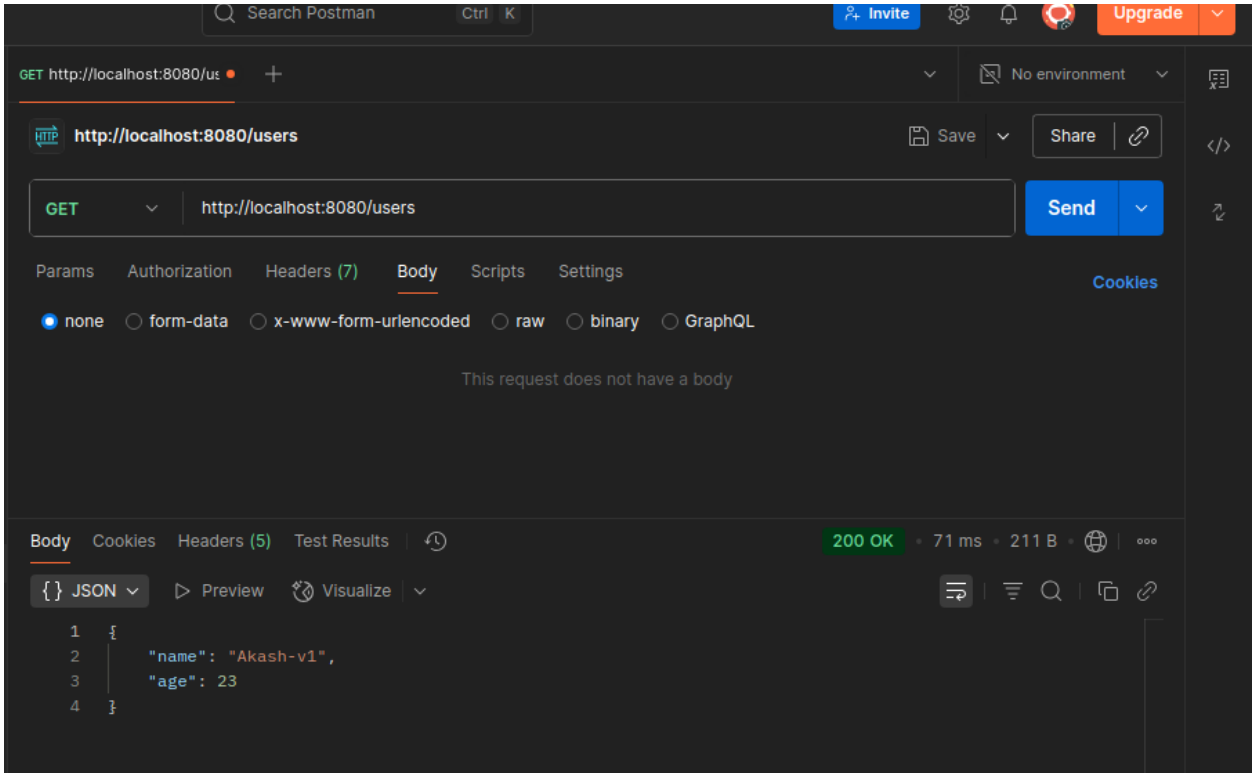


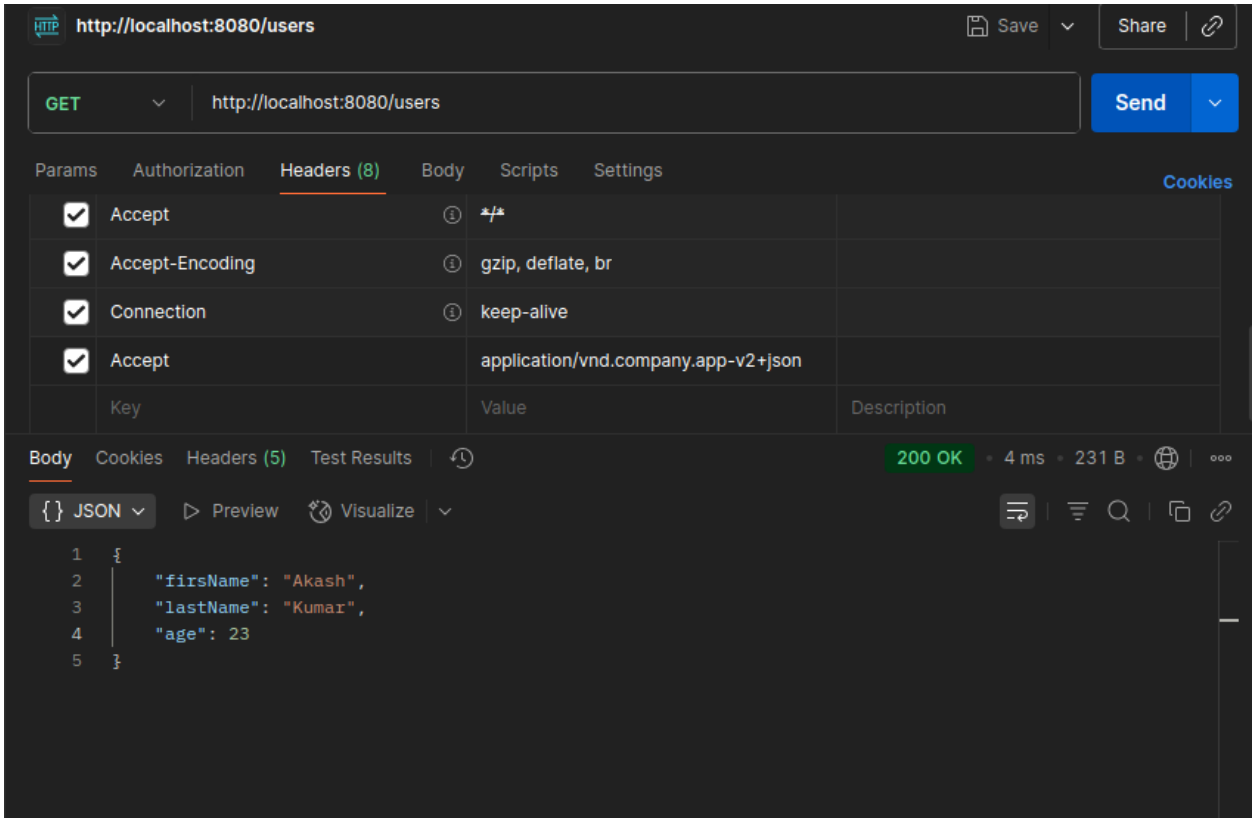

```java
@PostMapping("/dynamic/users")
public MappingJacksonValue createDynamicUser(@RequestBody UserEntity2 user) {
    MappingJacksonValue mappingJacksonValue = new MappingJacksonValue(user);

    SimpleBeanPropertyFilter propertyFilter = SimpleBeanPropertyFilter.filterOutAllExcept( ...propertyArray: "name", "email");
    FilterProvider filterProvider = new SimpleFilterProvider().addFilter( id: "userFilter", propertyFilter);
    mappingJacksonValue.setFilters(filterProvider);
    return mappingJacksonValue;
}
```

```java
@JsonFilter("userFilter")  2 usages
public class UserEntity2 {

    private String name;  3 usages
    private String email;  3 usages
    private String password;  3 usages

    public UserEntity2(String name, String email, String password) {  no usages
        this.name = name;
        this.email = email;
        this.password = password;
    }
```

Q5)Versioning Restful APIs Create 2 API for showing user details. The first api should return only basic details of the user and the other API should return more/enhanced details of the user, Now apply versioning using the following methods:

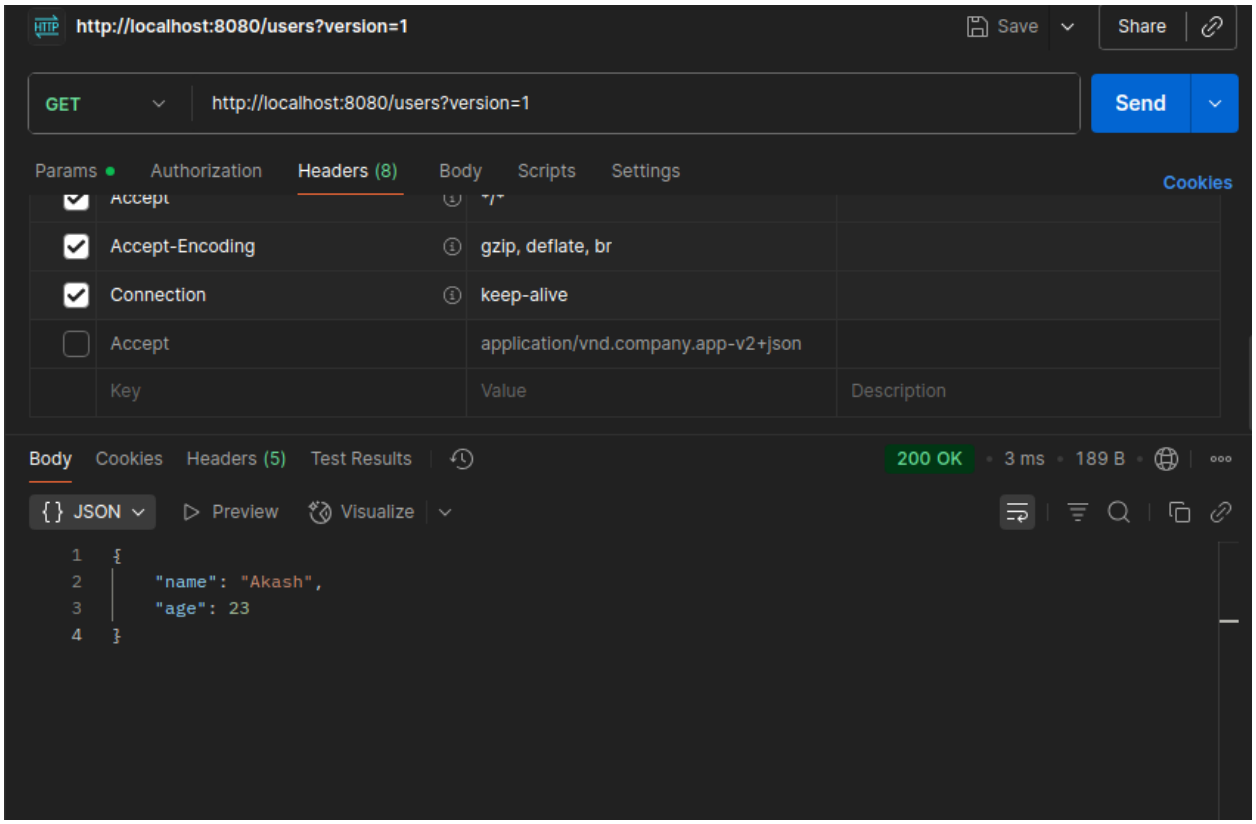
A) MimeType Versioning

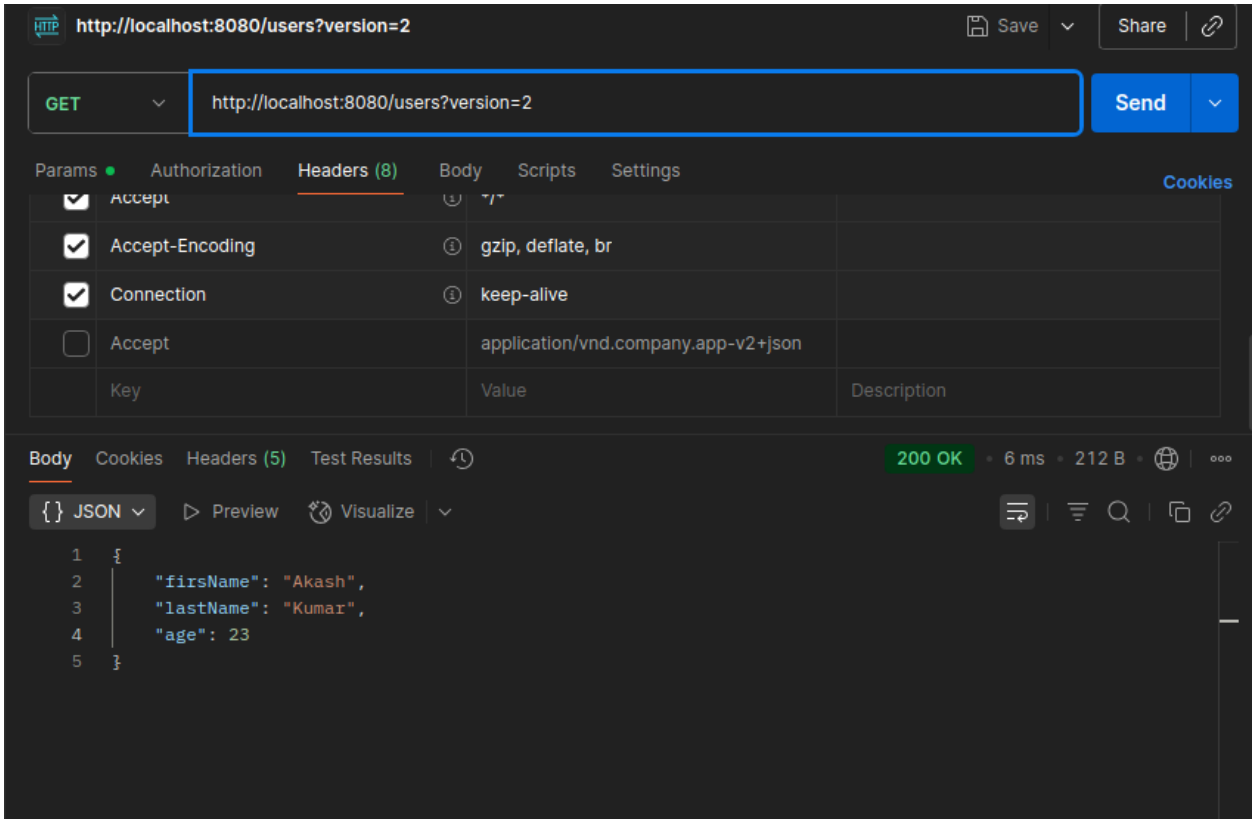





```java
import org.springframework.web.bind.annotation.RestController;

@RestController
public class VersionController {

    //MIME
    @GetMapping(value = "/users",produces = "application/vnd.company.app-v1+json")
    public UserV1 getUsers() {
        return new UserV1( name: "Akash-v1", age: 23);
    }

    @GetMapping(value = "/users", produces = "application/vnd.company.app-v2+json")
    public UserV2 getUsers2() {
        return new UserV2( firsName: "Akash", lastName: "Kumar", age: 23);
    }

}
```

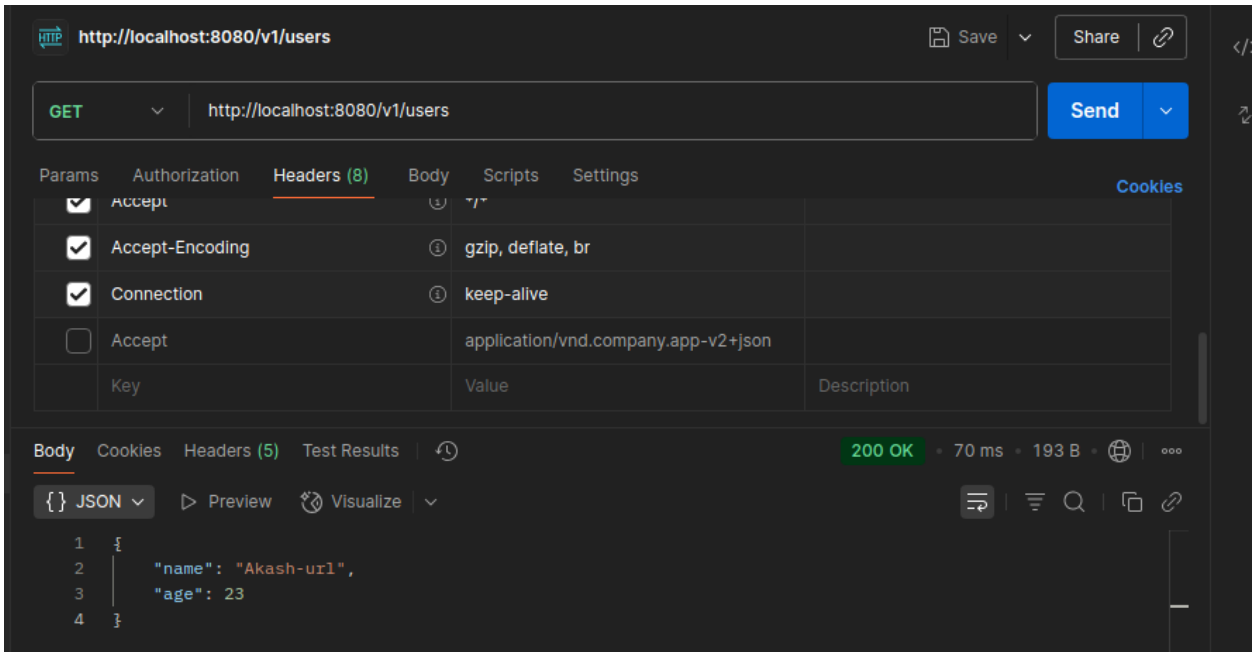
B) Request Parameter versioning

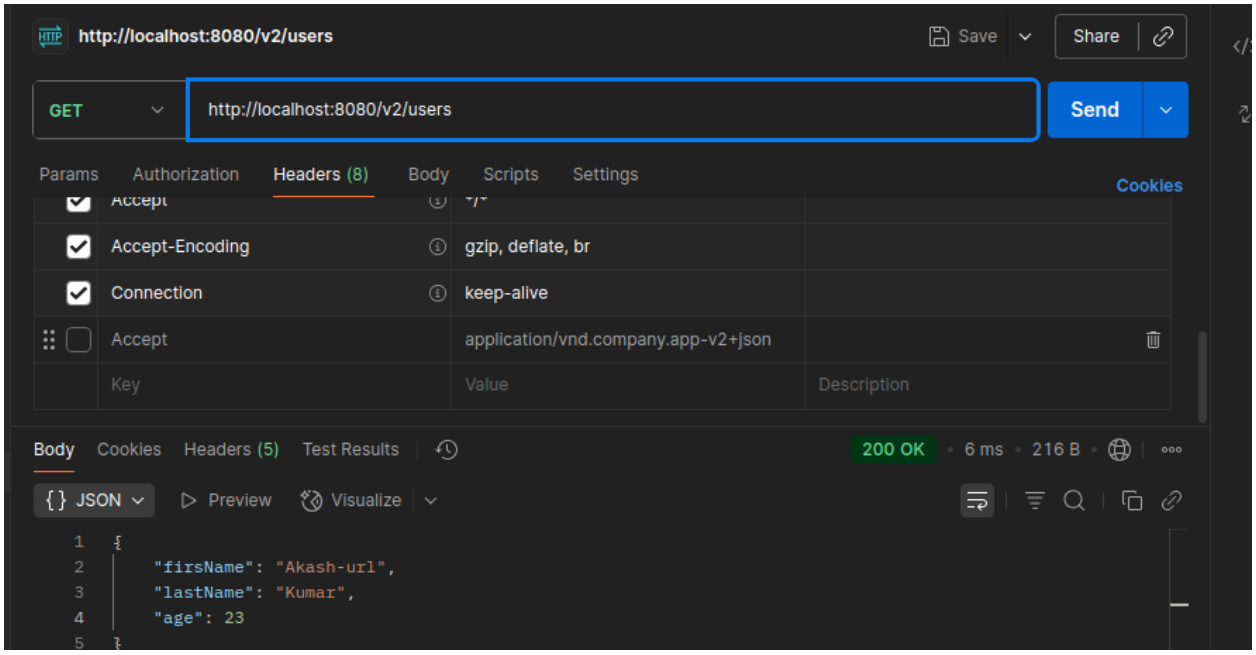




```java
@GetMapping(value = "/users", params = "version=1")
public UserV1 getUserParamV1() { return new UserV1( name: "Akash", age: 23); }

@GetMapping(value = "/users", params = "version=2")
public UserV2 getUserParamV2() { return new UserV2( firsName: "Akash", lastName: "Kumar", age: 23); }
```

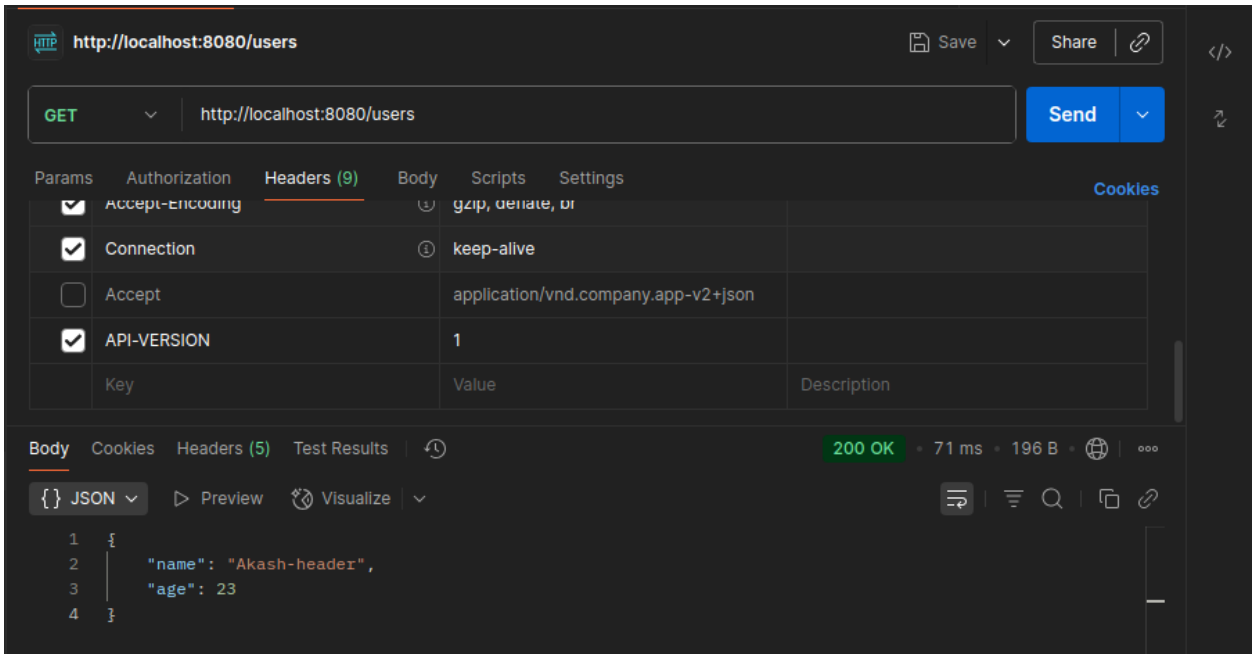
C) URI versioning

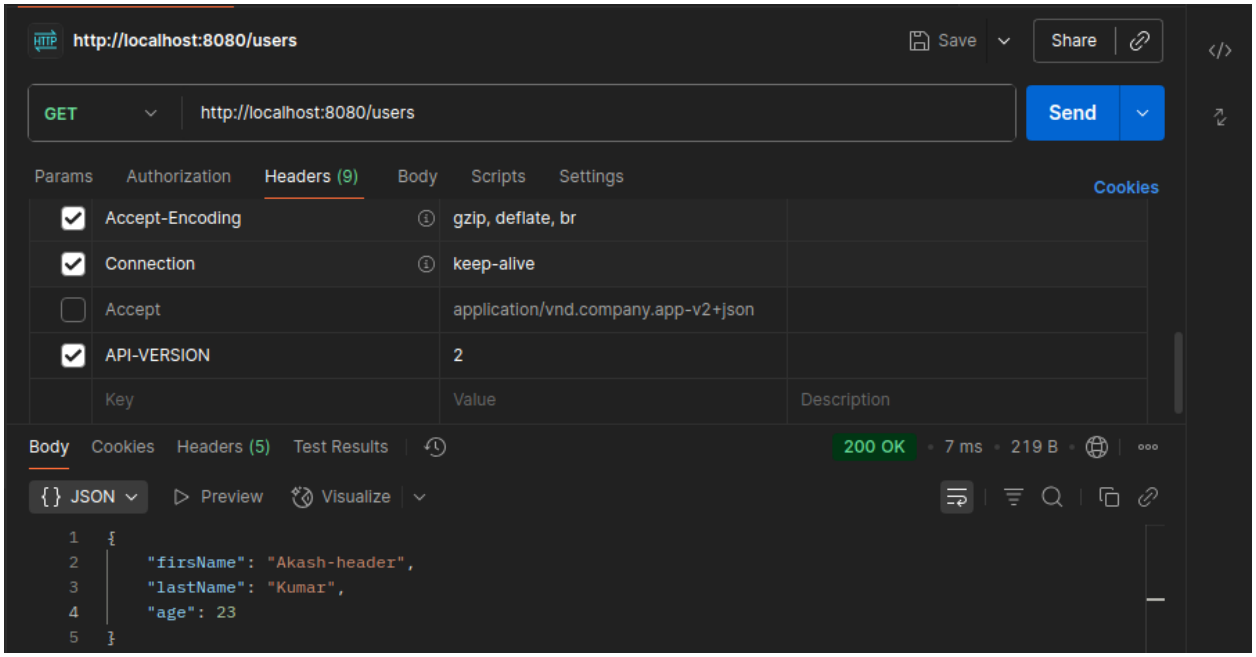





```java
//Url

@GetMapping(value = "/v1/users")
public UserV1 getUserUrlV1() { return new UserV1( name: "Akash-url", age: 23); }

@GetMapping(value = "/v2/users")
public UserV2 getUserUrlV2() { return new UserV2( firsName: "Akash-url", lastName: "Kumar", age: 23); }
```

D) Custom Header Versioning

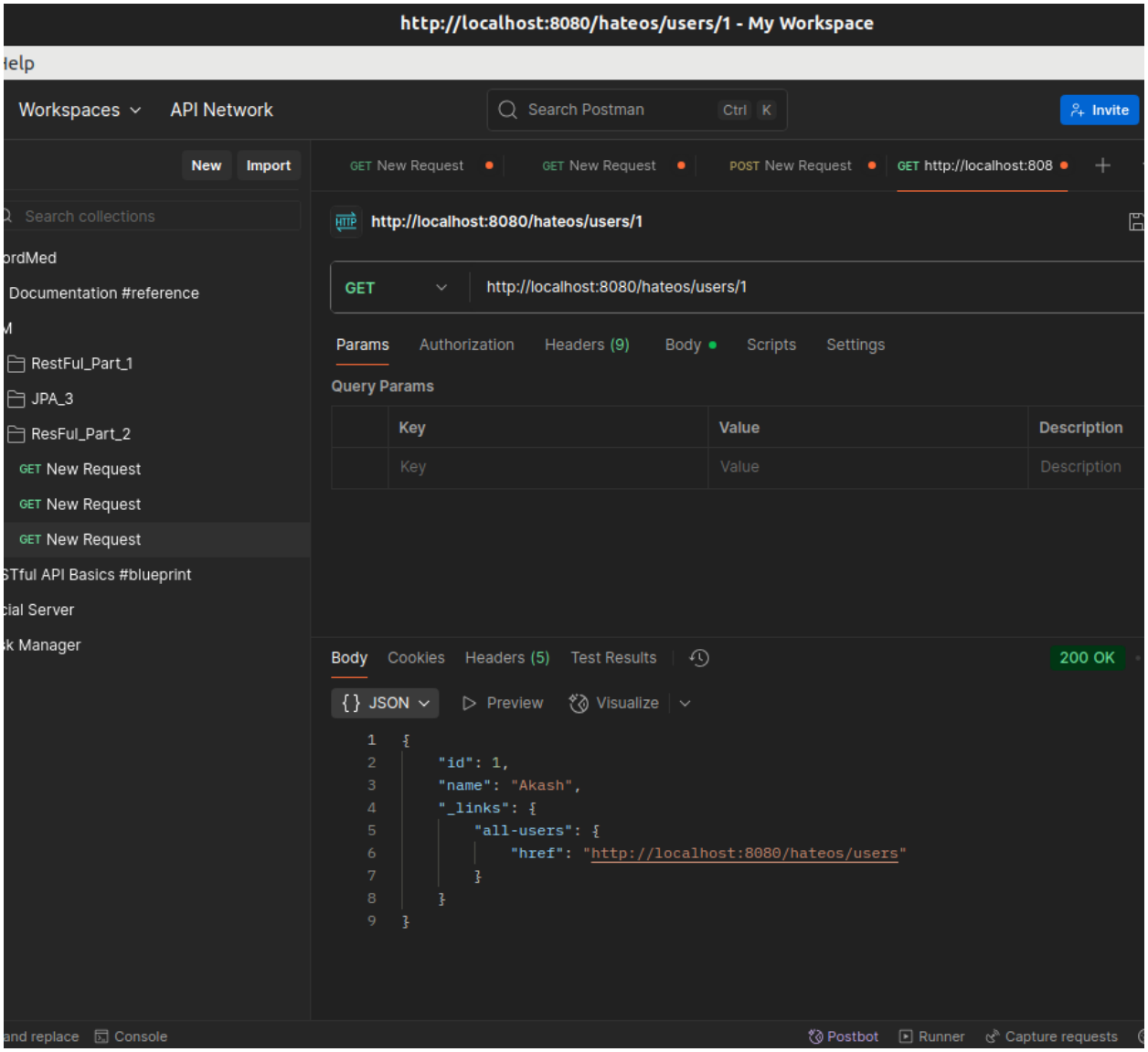




```java
//headers
@GetMapping(value = "/users",headers = "API-VERSION=1")
public UserV1 getUserHeaderV1() { return new UserV1( name: "Akash-header", age: 23); }

@GetMapping(value = "/users",headers = "API-VERSION=2")
public UserV2 getUserHeaderV2() { return new UserV2( firsName: "Akash-header", lastName: "Kumar", age: 23); }
```

Q6)HATEOAS

A) Configure hateoas with your springboot application. Create an api which returns User Details along with url to show all topics.



```java
import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.*;

@RestController
@RequestMapping("/hateos")
public class HateosController {
    @Autowired
    private  UserService userService;
    @GetMapping("/users")
    public List<User> getAllUsers() {
        return userService.getAllUsers();
    }
    @GetMapping("/users/{id}")
    public EntityModel<User> getUserById(@PathVariable int id) {
        User user = userService.findOne(id);
        EntityModel<User> entityModel = EntityModel.of(user);
        WebMvcLinkBuilder link = linkTo(methodOn(this.getClass()).getAllUsers());
        entityModel.add(link.withRel("all-users"));
         return entityModel;

    }
}
```